

Deciding the workloads

introduction

In this document, I outline the workload analysis requirements, detailing the types of workloads supported by the NPU, the key design considerations of the architecture, and the hardware constraints.

Motivation :

When I started this project, I looked at real car chips like the NVIDIA Drive Thor and Qualcomm Snapdragon Ride to see what the big companies are building. Seeing their speed numbers was honestly intimidating. Thor can handle up to 2,000 trillion operations per second, and Snapdragon Ride does around 300 to 700.

Looking at those massive numbers made me really nervous about my own NPU design. Because I'm focusing on self-driving car systems (ADAS), a scary thought hit me: what if my chip just isn't fast enough? In a moving car, if the computer is even a little bit late, it's not just a glitch—it's a huge danger.

For a car to hit the brakes in an emergency, the chip has to understand the camera and sensor data in less than 10 milliseconds. If the chip takes too long to do the math, the system slows down, and the car physically won't be able to react in time.

To fix this, I need to work backward from that strict time limit. Instead of just guessing, I have to figure out exactly how many calculation blocks my chip needs so it never misses a deadline and keeps the car perfectly safe

For what type of workload did I design this NPU?

When I studied a little about Hexagon, I understood that it uses a combination of a vector engine, a scalar engine, and tensor cores, which may be implemented using systolic-arraybased processing for matrix operations. This makes it a hybrid NPU architecture that is well suited to its target workloads.

However, if I were to target the same type of workloads, it would be difficult for me to build a machine that is at least viable, considering the complexity of such an architecture. Therefore, I made some assumptions and narrowed down the workload requirements.

For my NPU design, I need to consider two key factors: the required latency of the application and the total TOPS required by the model.

Most of the workloads I am considering are centered around ADAS applications. Therefore, I am focusing on workloads that are representative of ADAS systems while keeping the NPU architecture relatively simple and practical to implement.

Why I Chose ADAS (Advanced Driver Assistance Systems) as the Target Workload

I chose **ADAS (Advanced Driver Assistance Systems)** as the target workload because automotive systems require highly reliable and predictable computation. In safety-critical applications such as automobiles, the accelerator cannot simply provide high average performance; it must meet computational requirements within strict latency constraints.

The main design considerations that led me to choose ADAS are:

1. Latency

Latency is one of the most critical requirements in ADAS.

If an accelerator is responsible for processing an ADAS workload, it must complete the required computation within a specified time, regardless of the current operating state of the accelerator. The system should provide **predictable and bounded latency**, because delayed computation can directly affect the response of the vehicle.

2. Throughput

Throughput determines how much computation the NPU can perform within a given amount of time.

Depending on the data type, throughput can be expressed as:

- **TOPS** for INT8
- **TFLOPS** for FP8/FP16 and other floating-point formats

My goal is to design the NPU around the requirements of the **target ADAS workloads**, rather than maximizing theoretical peak performance alone.

For example, if a target application requires approximately **10 TOPS**, the NPU should be capable of providing the required computational throughput while still meeting the required latency constraints.

Therefore, the objective is not simply to build the largest possible accelerator, but to provide **sufficient computational capability for the intended workload within the required latency and power constraints.**

3. Computational Bound vs. Memory Bound

Another important consideration is the balance between **compute performance and memory performance.**

Ideally, I do not want the NPU to become unnecessarily **memory-bound** while there is still significant computational capacity available. At the same time, simply increasing compute resources is not useful if the memory system cannot supply data fast enough.

My design objective is therefore to find an appropriate balance between:

- Compute throughput
- Memory bandwidth
- Memory latency
- On-chip storage
- Data reuse
- Power consumption
- Area

However, I consider it acceptable for the system to become **temporarily memory-bound when operating at its maximum computational throughput**, provided that the memory system is being utilized efficiently and the overall system is achieving the required performance.

This is similar in principle to how processors can temporarily operate at higher performance levels, such as through boost frequencies, where the system operates near its performance limits for a limited period.

Overall Design Philosophy

Therefore, my objective is not to eliminate every possible memory bottleneck or maximize peak TOPS in isolation.

Instead, I want to design the NPU so that **the compute and memory systems are balanced for the characteristics of the target ADAS workloads.**

The key question I am trying to answer through this project is:

What is the optimal balance between compute capability, memory bandwidth, latency, power, and area for an NPU designed specifically for ADAS workloads?

This workload-driven approach is the basis for the architectural decisions I am making in my project.

I want to classify the workload into two types.

CNN (Convolutional Neural Networks) are really good at image processing, such as object detection, lane detection, and other vision-related tasks.

But there is a problem: **how does the car make the decision about when to turn, when to stop, or what action to take?** And what about the reliability of that decision?

Maybe we also need to support a **voice assistant**. These are additional workloads that we need to fulfill, which means I need to be able to run **Transformer models** on the NPU as well. So, the NPU needs to support both types of workloads.

i make the EXEL sheet for the workload table it is difficult to place the table here in the form of PDF in the exel sheet there are two section for CNN and fro transformer models both in one exel sheet i am also adding some of the reference here for the workload that i consider

this is for CNN

References

1. **SSD (ResNet-50) - MLPerf Automotive**

- SSD: Single Shot MultiBox Detector — Liu et al., 2016

2. **DeepLabv3+ (ResNet-50) MLPerf Automotive**

- DeepLabv3+: Encoder-Decoder with Atrous Separable Convolution — Chen et al., 2018

2. **BEVFormer-Tiny MLPerf Automotive**

• [BEVFormer](#) — Li et al., 2022

4. **PointPillars** - [PointPillars](#) — Lang et al., 2019 • [PointDistiller](#) — arXiv
5. **CenterPoint-Voxel** - [CenterPoint](#) — Yin et al., 2021 • [VoxelNeXt](#) — Chen et al., 2023
6. **VoxelNeXt** - [VoxelNeXt](#) — Chen et al., 2023
7. **YOLOv8n** - [Ultralytics YOLOv8 Documentation](#) • [YOLOv8](#) — arXiv
8. **YOLOv8m** - [Ultralytics YOLOv8 Documentation](#) • [YOLOv8](#) — arXiv
9. **YOLOv5x**- [YOLOv5](#) — Ultralytics
10. **YOLOP**- [YOLOP: You Only Look Once for Panoptic Driving Perception](#) — Wu et al., 2021
11. **BEVDet-Tiny** [BEVDet](#) — Huang et al., 2021
12. **BEVDet-Base** [BEVDet](#) — Huang et al., 2021
13. **Tesla HydraNet-style Stack** [Odema et al., 2024](#) — arXiv • [Tesla AI Day 2021](#)
14. **Tesla Occupancy-Network Trunk** [Odema et al., 2024](#) — arXiv
• [Tesla AI Day 2021](#)

above all the model are in the EXL table

For transformer model :

References

A — Reasoning / Decision Workloads

1. **Llama-3.2-3B-Instruct** - [Meta](#) — [Llama 3.2 Model Card](#)
2. **Llama-3.1-8B-Instruct** - [Meta](#) — [Llama 3.1 Model Card](#)
3. **Llama-3.1-70B-Instruct** - [Meta](#) — [Llama 3.1 Model Card](#)
4. **Mixtral-8x7B** - [Mixtral of Experts](#) — Jiang et al., 2024 • [MELINOE](#) — arXiv:2602.11192
5. **DeepSeek-V3 / R1** - [DeepSeek-V3 Technical Report](#)

B — Sensor / Perception Workloads

6. **Extended Kalman Filter** - [A New Approach to Linear Filtering and Prediction Problems](#) — Kalman, 1960
7. **PointNet** - [PointViG](#) — arXiv:2407.00921
8. **PointNet++ (MSG)** - [PointViG](#) — arXiv:2407.00921
9. **PointPillars** - [PointDistiller](#) — arXiv:2205.11098

- [PointPillars](#) — Lang et al., CVPR 2019

C — Edge LLM Workloads

10. [Llama-3.2-1B \(INT4\)](#) - Meta — Llama 3.2 Model Card
11. [Qwen2.5-7B-Instruct on Xiaomi 15 Pro \(MNN\)](#) - CORE — [arXiv:2510.15455](#)
12. [Llama-2-7B](#) - Koh & Yang — [arXiv:2511.01695](#)

D — Voice Workloads

13. **Whisper Large-v3**
 - [Groq](#) — Whisper Large-v3 Model Card
 - [Whisper](#) — [arXiv:2410.16330](#)
14. **FastConformer-Transducer** - Typhoon ASR Real-time — [arXiv:2601.13044](#)
15. **WaveNet**
 - [WaveNet: A Generative Model for Raw Audio](#) — DeepMind
 - [USPTO Patent 11,705,107](#) — Real-time Neural Text-to-Speech
16. **Inworld TTS-1 / TTS-1-Max** - Inworld — TTS-1 Technical Report

Deciding the Headroom

Snapdragon Ride provides around **300–750 TOPS of INT8 performance**, while NVIDIA DRIVE AGX Thor provides around **1,000 TOPS of INT8 performance** and also supports high floatingpoint performance such as FP8. These are very large numbers compared with the workloads I have analyzed above.

If my objective is only to run two models in parallel — one CNN-based model and one Transformer-based model — the required computational capacity still appears considerably smaller than the peak performance of these platforms.

So the question is:

Why do they design the system with such a large amount of computational capacity?

One important answer is the **lifetime of the vehicle**.

If someone buys a car, they may use it for **10 years or even longer**. The computing system inside the vehicle can also continue operating for a very long time as long as there is no physical

or electrical failure. Therefore, nobody wants to replace the entire computing system simply because the computational requirements have increased.

The important point is that the computational system does not usually become obsolete because the silicon itself has degraded. Instead, it can become obsolete because the **workload changes drastically over time**.

For example, I did a small case study on the **Snapdragon 730G**.

Qualcomm started designing this SoC around **2019**. About a year later, Vivo implemented it into one of its smartphones, and I bought that smartphone in **2021**. I am still using that smartphone today, and the hardware is still working fine.

This shows an important point: the SoC was designed several years before I actually purchased the device, and it was expected to remain useful for years after that.

So, when designing an automotive computing system, we cannot look only at the computational requirements of today's models.

We have to consider the **future workload**.

This is what I mean by **headroom design**.

The goal is not simply to provide enough computational performance for the current CNN or Transformer models. The system should have enough additional computational capability to accommodate the **drastic increase and evolution of AI workloads over the lifetime of the vehicle**.

AI workloads are increasing rapidly, and the models are also becoming larger and more computationally demanding.

Therefore, the large TOPS numbers seen in modern automotive platforms may not necessarily mean that the current workload requires all of that computation. A significant part of that computational capacity can be viewed as **headroom for future workloads, software updates, newer models, and increasing AI complexity over the lifetime of the vehicle**.

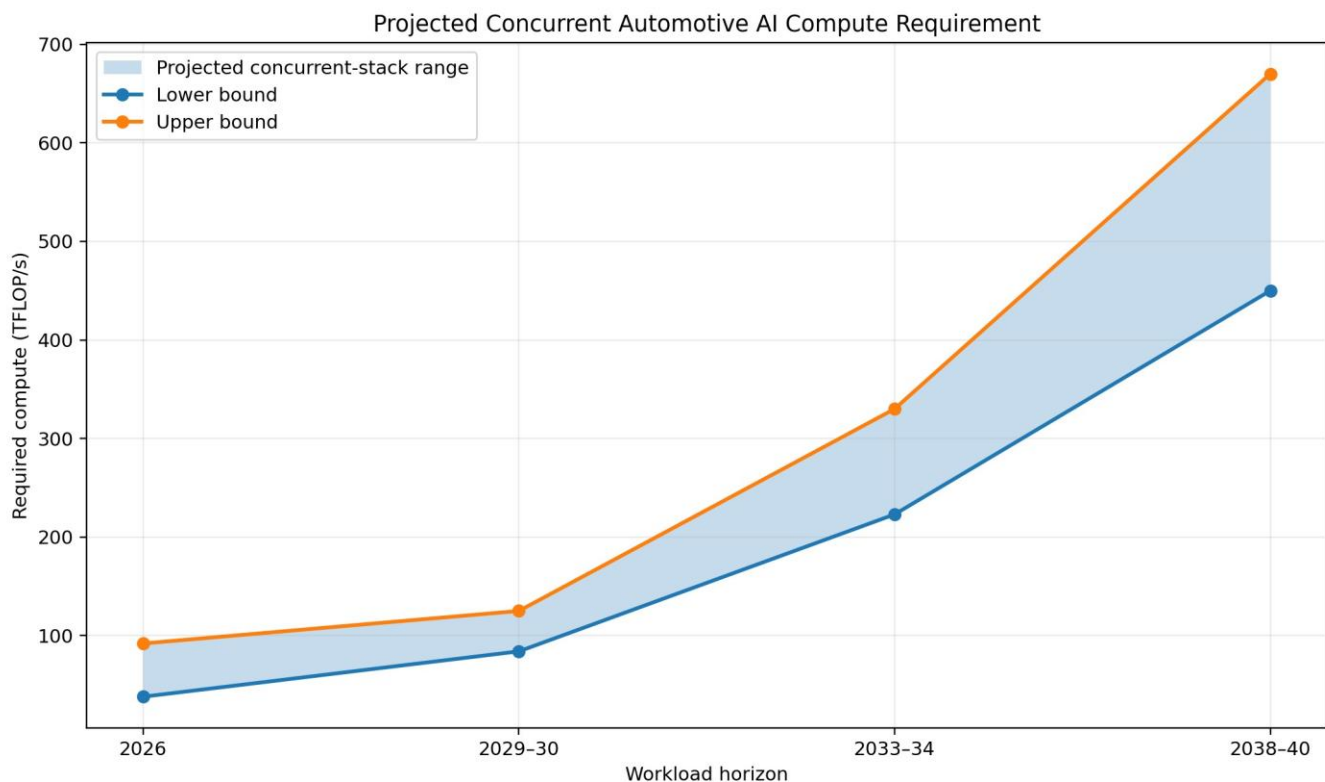
how much head room should i need :

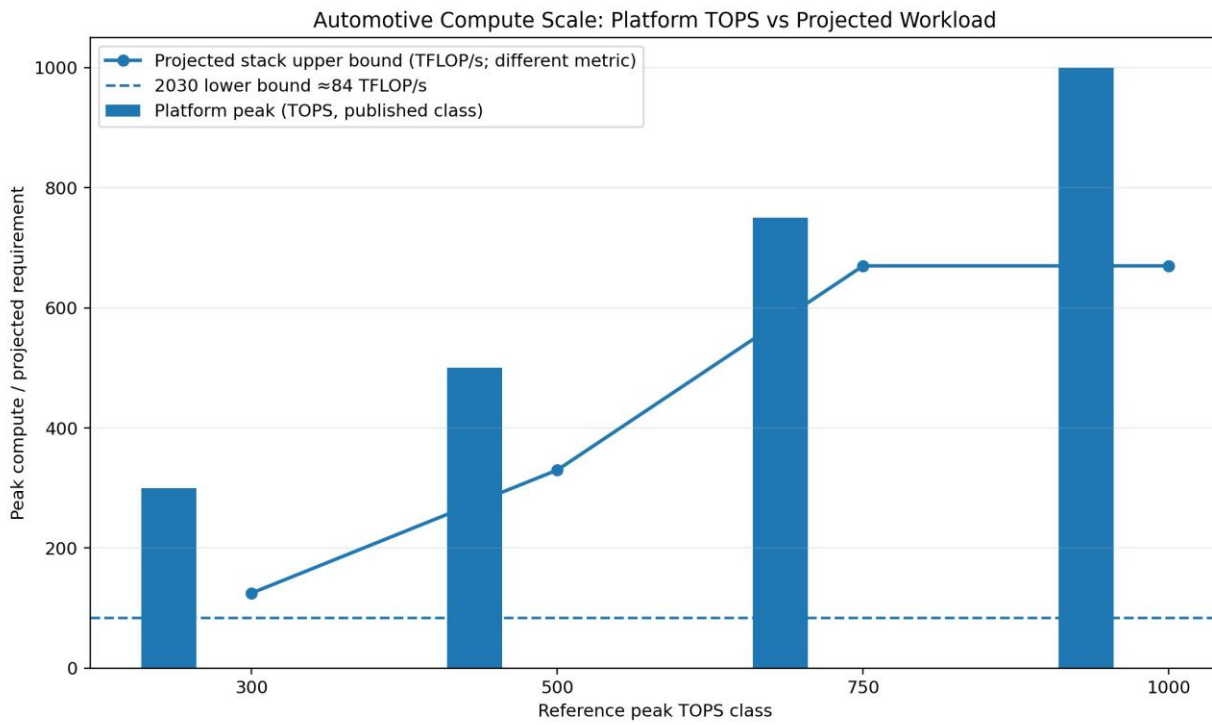
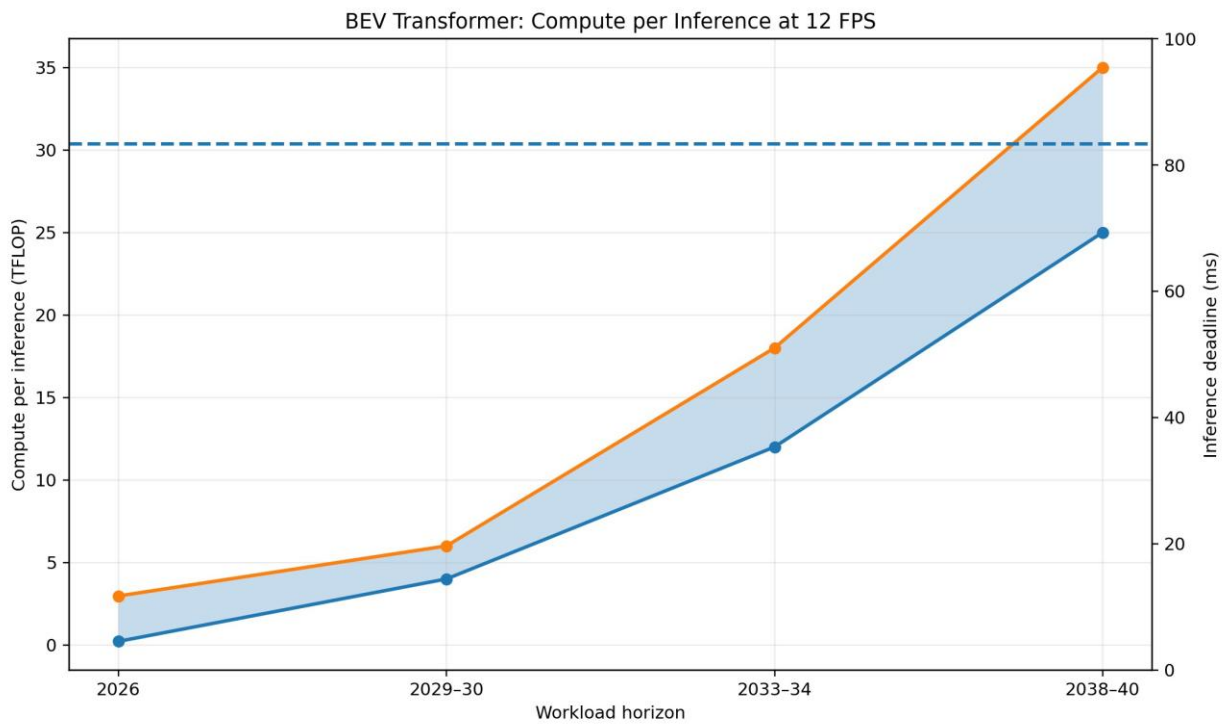
$$\text{Headroom} = \frac{\text{Available Performance}}{\text{Required Performance}} - 1$$

ADAS Future Workload Graphs

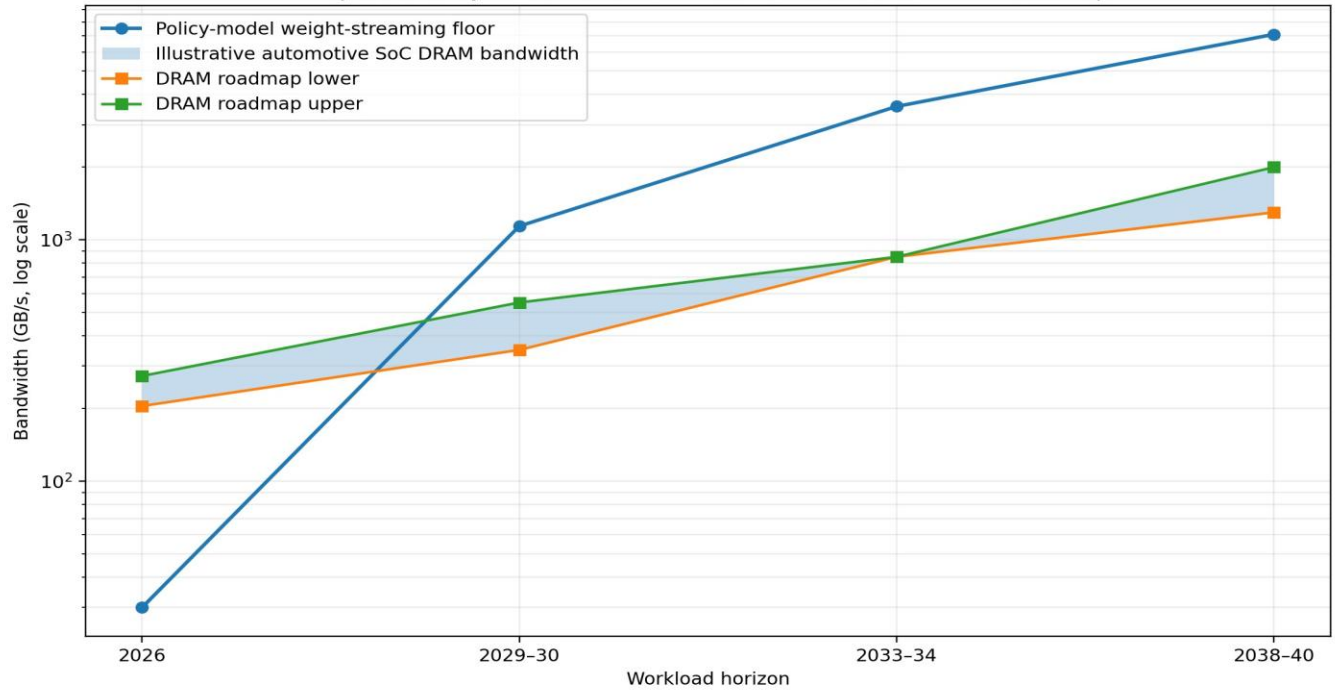
Important:

- Values marked [PROJECTED] in the source are the document's own forward projections.
- They are planning scenarios, not vendor commitments.
- The source explicitly states that future figures should be sensitivity-tested and revisited.
- Peak INT8 TOPS and TFLOP/s are different metrics. The platform-comparison plot is therefore only a scale comparison, not a claim of equivalent performance.
- The BEV/occupancy Transformer workload is modeled at 12 FPS, implying an 83.3 ms task period in the source methodology.

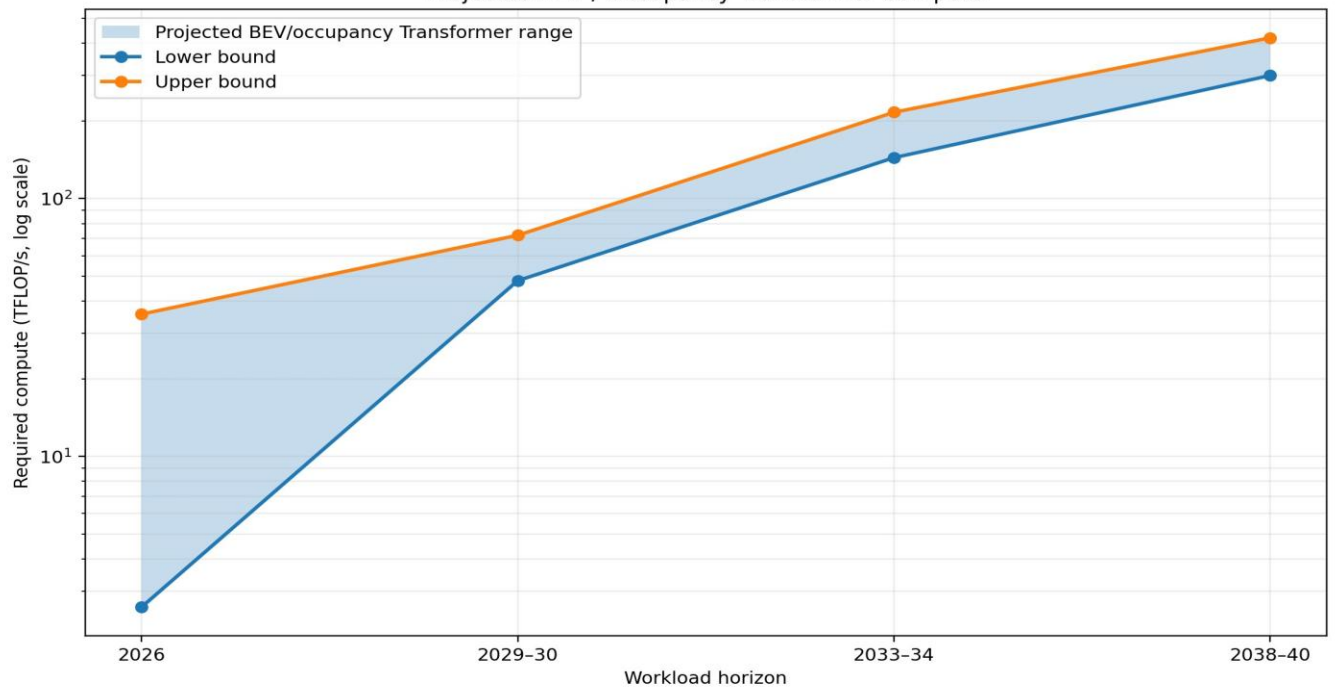




Projected Policy-Model Bandwidth Floor vs Automotive DRAM Roadmap



Projected BEV / Occupancy Transformer Compute



Methods	Image Size	#param.	GFLOPs	Modality	mAP $\uparrow$	mATE $\downarrow$	mASE $\downarrow$	mAOE $\downarrow$	mAVE $\downarrow$	mAAE $\downarrow$	NDS $\uparrow$	FPS
VoxelNet [56]	-	-	-	LiDAR	0.564	-	-	-	-	-	0.648	-
PointPillar [56]	-	-	-	LiDAR	0.503	-	-	-	-	-	0.602	-
CenterNet [59]	-	-	-	Camera	0.306	0.716	0.264	0.609	1.426	0.658	0.328	-
FCOS3D [49]	1600 $\times$ 900	52.5M	2,008.2	Camera	0.295	0.806	0.268	0.511	1.315	0.170	0.372	1.7
DETR3D [51]	1600 $\times$ 900	51.3M	1,016.8	Camera	0.303	0.860	0.278	0.437	0.967	0.235	0.374	2.0
PGD [50]	1600 $\times$ 900	53.6M	2,223.0	Camera	0.335	0.732	0.263	0.423	1.285	0.172	0.409	1.4
BEVDet-Tiny	704 $\times$ 256	53.7M	215.3	Camera	0.312	0.691	0.272	0.523	0.909	0.247	0.392	15.6
BEVDet-Base	1600 $\times$ 640	126.6M	2,962.6	Camera	0.393	0.608	0.259	0.366	0.822	0.191	0.472	1.9
FCOS3D $\dagger$ # [49]	1600 $\times$ 900	-	-	Camera	0.343	0.725	0.263	0.422	1.292	0.153	0.415	-
DETR3D $\dagger$ [51]	1600 $\times$ 900	51.3M	-	Camera	0.349	0.716	0.268	0.379	0.842	0.200	0.434	-
PGD $\dagger$ § [50]	1600 $\times$ 900	53.6M	-	Camera	0.369	0.683	0.260	0.439	1.268	0.185	0.428	-
BEVDet-Base §	1600 $\times$ 640	126.6M	-	Camera	0.397	0.595	0.257	0.355	0.818	0.188	0.477	-

References

- MLPerf Automotive — MLCommons
- MLPerf Automotive v5.0
- NVIDIA DRIVE AGX Thor — Safety Report
- NVIDIA DRIVE AGX Thor Platform for Developers
- NVIDIA DRIVE Thor Announcement
- Qualcomm Snapdragon Ride Platform Qualcomm —
- Snapdragon Ride / Automotive Compute imec
- Automotive Chiplet Program imec Automotive
- Technologies

This graph contains a very broad prediction, but for this project, I am considering the workload prediction only for 2030, not beyond that.

Headroom calculation and Tops calculation :

HeadRoom Calculation & TFLOPS Calculation:

For INT8: this is for CNN

I consider here 2D & 3D Detection in parallel.

For 2D Best Model which I found is SSD(ResNet-50)
Deeplabv3 + (ResNet-50)

For 3D $\rightarrow$ BEVDet-Base (BEV-high accuracy)

For Reliability I will use both at same time:

SSD(ResNet-50) - 3227 - GOPS

BEVDet-Base - 2962.6 - GOPS

} These are the
max value from my
workload analysis.

For BEVDet-Base the Frame rate is
18 FPS

I am taking 18 Frame's per second

So. total TOPS is:-

$$18 \times (3.227 + 2.962)$$

$$= 111.402 \text{ TOPS} - \text{in the worst case.}$$

By the chart which I added in above file I decided
36.6% of head_{room} with the current ~~740~~ TOPS.

$\therefore$ The Extra TOPS are:- $111.402 \times 36.6 \approx 39$

$$\text{So. Total TOPS} = \boxed{150 \text{ TOPS}}$$

Calculation For FP8: Required Compute = $\frac{\text{operation per inference}}{\text{Allowed Inference Latency}}$

Workload:

unifide multitask r- : 1800-2400 GFLOPS - Rate used = 15Hz
Precision

BEV / occupancy Transformer: 4000-6000 GFLOPS = 12Hz

LIDAR / radar sparse Fusion: 350-500 GFLOPS - 10Hz

Redundant 2D Detector: 15-150 GFLOPS - 30Hz

Occupancy decoder: 300-500 GFLOPS - 15Hz

So, total: Minimum: $27 + 48 + 3.5 + 0.45 + 4.5 = 83.45 \text{ TFLOPS}$
Maximum: $36 + 72 + 5 + 4.5 + 7.5 = 125 \text{ TFLOPS}$

Take a mid point: 104 TOPS

~~With our 36 y. a head~~
For this case I will take head room of 50%.
as it Transformer model increase rapidly with lot of
Parameter $\therefore$

$$\text{Total TFLOPS} = 104 \times \frac{50}{100} + 104 = 156 \text{ TFLOPS}$$

in this hardware i am not adding FP4 support as it is
completely new data type I don't have lot of information
about. ~~So~~ maybe by adding this will save
lot of energy and memory. Like NVIDIA Thor
support 2000 TFLOP of FP4 support
For my NPU supported data type ~~should~~ are
INT8 & FP8

THIS calculation gives result of TOPS for int8 is 150 and for FP8 156

Conclusion

This file provides a way to perform quantitative analysis by directly observing the workload and calculating the total TOPS by summing the net parallel workload. The computational requirements of the decision-making algorithm are not covered, as they are outside the scope of this project.

For this project, I am considering a computational capability of **150 TOPS for INT8** and **156 TFLOPS for FP8**.